

Session 12 : TP — Système Solaire & Intégration RK4

Objectifs du TP

- Simuler un **système solaire** complet avec la gravité de Newton.
- Comprendre **pourquoi** l'intégration d'Euler ne suffit pas pour les orbites.
- Implémenter **Runge-Kutta 4 (RK4)** pour une intégration stable sur les orbites.
- Découvrir le **sub-stepping** — diviser le pas de temps pour gagner en précision.
- Trouver la **vitesse initiale** pour une orbite circulaire parfaite.
- **Bonus** : ajouter la gravité mutuelle entre planètes (N-body).

Contexte

Depuis la Session 1, on intègre les équations du mouvement avec **Euler** : $v + = a \cdot dt$, puis $x + = v \cdot dt$. C'est simple, rapide, et suffisant pour des particules amorties (Session 9) ou du tissu (Session 8). Mais pour les **orbites planétaires**, Euler accumule de l'erreur à chaque pas — les planètes dérivent de leur orbite et finissent par s'échapper. Ce TP introduit **RK4**, une méthode d'intégration qui "sonde" le pas de temps à 4 endroits pour estimer la trajectoire avec une précision $O(dt^4)$.

Partie 1 : La Gravitation Universelle

Loi de gravitation de Newton

Deux masses m_1 et m_2 séparées par une distance r s'attirent avec une force :

$$\vec{F} = G \cdot \frac{m_1 m_2}{r^2} \cdot \hat{u}$$

où G est la constante gravitationnelle, r la distance entre les centres, et $\hat{u}$ la direction de l'attraction.

L'accélération subie par un objet de masse m sous l'attraction d'une masse M est :

$$\vec{a} = G \cdot \frac{M}{r^2} \cdot \hat{u}$$

Notez que la masse m de l'objet **s'annule** — l'accélération ne dépend que de la masse attractive M et de la distance r .

Pourquoi on ne ressent pas l'attraction entre deux personnes ?

Parce que $G = 6.674 \times 10^{-11} \text{N} \cdot \text{m}^2/\text{kg}^2$ est **minuscule**. Deux personnes de 70 kg à 1 m l'une de l'autre s'attirent avec une force de $\sim 3 \times 10^{-7} \text{N}$ — moins qu'un grain de poussière. Il faut une masse **planétaire** pour que la gravité devienne perceptible.

Le Soleil comme source unique

Dans ce TP, on fait une approximation : le Soleil est **fixe** au centre et est la seule source de gravité. Les planètes sont attirées par le Soleil mais ne s'attirent pas entre elles (sauf bonus).

Accélération vers le Soleil

Pour une planète à la position $\vec{r}$ (le Soleil est à l'origine), l'accélération est :

$$\vec{a} = -G \cdot \frac{M_{\text{soleil}}}{\|\vec{r}\|^2} \cdot \hat{r} = -G \cdot \frac{M_{\text{soleil}}}{\|\vec{r}\|^3} \cdot \vec{r}$$

En code :

```
function getAcceleration(position) {
    const r2 = position.lengthSq(); // ||r||²
    if (r2 < 5) return new THREE.Vector3(0,0,0); // singularité évitée
    const fMag = (G * SUN_MASS) / r2; // G·M / r²
    return position.clone().negate().normalize().multiplyScalar(fMag);
}
```

💡 Pourquoi le test $r^2 < 5$?

Si une planète atteint le centre du Soleil ($r = 0$), l'accélération devient infinie ($\frac{1}{r^2} \rightarrow \infty$). On ajoute un **seuil de sécurité** : si la distance est trop petite, on annule l'accélération. C'est une rustine — dans un vrai moteur, on gèrerait la collision avec la surface du Soleil.

Partie 2 : La Vitesse Orbitale

Orbite circulaire — équilibre force gravitationnelle / force centripète

Pour qu'une planète orbite en **cercle parfait**, il faut que la force gravitationnelle exactement compense la force centripète :

$$G \cdot \frac{M}{r^2} = \frac{v^2}{r}$$

En simplifiant :

$$v = \sqrt{G \cdot \frac{M}{r}}$$

C'est la **vitesse orbitale** — la vitesse tangentielle qu'il faut donner à une planète à la distance r pour qu'elle orbite en cercle.

💡 Direction de la vitesse initiale

La vitesse doit être **perpendiculaire** à la direction du Soleil. Si la planète est sur l'axe x (position $(r, 0, 0)$), la vitesse doit être sur l'axe z ou y :

```
pos: new THREE.Vector3(distWorld, 0, 0),
vel: new THREE.Vector3(0, 0, -vOrbit), // perpendiculaire, sens trigonométrique
```

Les trois cas selon la vitesse initiale

Vitesse	Trajectoire	Forme
$v < v_{\text{orb}}$	La planète tombe vers le Soleil	Ellipse décadente / crash
$v = v_{\text{orb}}$	Orbite circulaire parfaite	Cercle
$v > v_{\text{orb}}$	La planète s'éloigne	Ellipse ou hyperbole (évasion)

Table 1: Trois régimes selon la vitesse initiale. En dessous de v_{orb} , la planète spirale vers le Soleil. Au-dessus, elle s'échappe. À v_{orb} exactement, l'orbite est circulaire.

💡 Tester dans le TP

Dans la solution, changez manuellement la vitesse de la Terre : multipliez v_{orb} par 0.8 (la Terre spirale vers le Soleil) ou par 1.3 (la Terre s'échappe en ellipse large). C'est une excellente façon de **sentir** la physique.

Partie 3 : Pourquoi Euler ne Suffit Pas

Le problème d'Euler sur les orbites

L'intégration d'Euler **explicite** met à jour la vitesse puis la position :

$$v_{n+1} = v_n + a \cdot dt$$

$$x_{n+1} = x_n + v_{n+1} \cdot dt$$

Le problème : sur une orbite, la force **change constamment de direction**. Euler suppose que la vitesse est **constante** sur tout le pas dt . À chaque pas, il "dérage" légèrement tangentiellement — l'erreur s'accumule et l'orbite **grandit** progressivement. La planète gagne de l'énergie à chaque révolution.

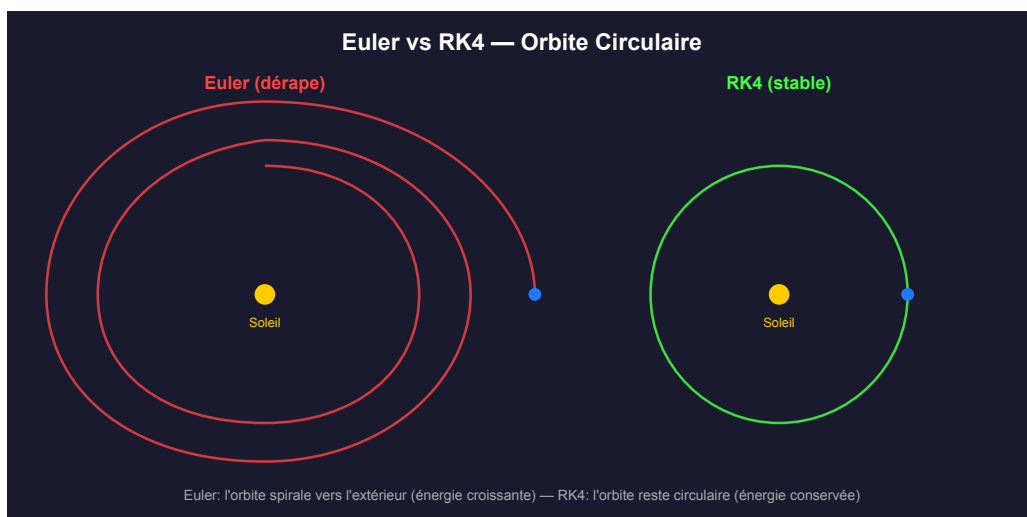


Figure 1: Euler vs RK4 sur une orbite circulaire. Avec Euler (rouge), l'orbite spirale vers l'extérieur — l'énergie augmente à chaque tour. Avec RK4 (vert), l'orbite reste stable même après des dizaines de révolutions.

! Euler n'est pas mauvais — il est mal adapté

Euler fonctionne parfaitement pour des systèmes **amortis** (particules avec drag, tissu) où l'énergie se dissipe. Mais pour un système **conservatif** comme les orbites (pas de friction), l'erreur d'Euler ne se dissipe jamais — elle s'accumule. C'est pourquoi on a besoin d'une méthode qui **respecte l'énergie**.

Partie 4 : Runge-Kutta 4 (RK4)

L'idée de RK4

Au lieu d'évaluer l'accélération **une seule fois** par pas (comme Euler), RK4 l'évalue à **quatre endroits** du pas de temps et fait une moyenne pondérée :

1. k_1 : évaluer au début du pas (t)
2. k_2 : évaluer au milieu du pas ($t + dt/2$), en utilisant k_1
3. k_3 : évaluer au milieu du pas ($t + dt/2$), en utilisant k_2
4. k_4 : évaluer à la fin du pas ($t + dt$), en utilisant k_3

La mise à jour finale :

$$y_{n+1} = y_n + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}$$

💡 L'analogie du GPS

Euler, c'est conduire en regardant uniquement le rétroviseur : on suppose qu'on va dans la même direction qu'il y a dt secondes, puis on corrige la vitesse. RK4, c'est regarder la route à 4 endroits devant soi et faire une moyenne — beaucoup plus précis.

RK4 appliqué à la gravité

État et dérivée

Pour la gravité, l'état d'une planète est la paire $(\vec{r}, \vec{v})$ (position + vitesse). La dérivée de cet état est $(\vec{v}, \vec{a})$ — la vitesse dérive la position, l'accélération dérive la vitesse.

RK4 calcule donc 4 paires (k_x, k_v) :

1. $k_1 = (\vec{v}, \vec{a}(\vec{r}))$ — état actuel
2. $k_2 = \left(\vec{v} + k_{1_v} dt/2, \vec{a}(\vec{r} + k_{1_x} dt/2) \right)$ — milieu avec k_1
3. $k_3 = \left(\vec{v} + k_{2_v} dt/2, \vec{a}(\vec{r} + k_{2_x} dt/2) \right)$ — milieu avec k_2
4. $k_4 = \left(\vec{v} + k_{3_v} dt, \vec{a}(\vec{r} + k_{3_x} dt) \right)$ — fin avec k_3

Mise à jour :

$$\vec{r} + = \frac{k_{1_x} + 2k_{2_x} + 2k_{3_x} + k_{4_x}}{6}$$

$$\vec{v} + = \frac{k_{1_v} + 2k_{2_v} + 2k_{3_v} + k_{4_v}}{6}$$

⚠ Ne pas modifier l'état pendant le calcul !

Les 4 étapes de RK4 évaluent l'accélération à des positions **intermédiaires** — il ne faut pas modifier `p.pos` et `p.vel` pendant le calcul. Utilisez des `.clone()` pour créer des copies des positions/vitesses intermédiaires. La mise à jour finale se fait **à la fin**, une seule fois.

RK4 en JavaScript.

```
function updatePhysicsRK4(p, dt) {
    const x0 = p.pos.clone();
    const v0 = p.vel.clone();

    // k1 – évaluer à t
    const k1_v = getAcceleration(x0).multiplyScalar(dt);
    const k1_x = v0.clone().multiplyScalar(dt);

    // k2 – évaluer à t + dt/2
    const x2 = x0.clone().addScaledVector(k1_x, 0.5);
    const v2 = v0.clone().addScaledVector(k1_v, 0.5);
    const k2_v = getAcceleration(x2).multiplyScalar(dt);
    const k2_x = v2.multiplyScalar(dt);

    // k3 – évaluer à t + dt/2
    const x3 = x0.clone().addScaledVector(k2_x, 0.5);
    const v3 = v0.clone().addScaledVector(k2_v, 0.5);
    const k3_v = getAcceleration(x3).multiplyScalar(dt);
    const k3_x = v3.multiplyScalar(dt);

    // k4 – évaluer à t + dt
    const x4 = x0.clone().add(k3_x);
    const v4 = v0.clone().add(k3_v);
    const k4_v = getAcceleration(x4).multiplyScalar(dt);
    const k4_x = v4.multiplyScalar(dt);

    // Mise à jour finale
    p.vel.add(k1_v.addScaledVector(k2_v, 2)
              .addScaledVector(k3_v, 2)
              .add(k4_v).divideScalar(6));
    p.pos.add(k1_x.addScaledVector(k2_x, 2)
              .addScaledVector(k3_x, 2)
              .add(k4_x).divideScalar(6));
}
```

Partie 5 : Le Sub-Stepping

Pourquoi le sub-stepping ?

Même avec RK4, un **trop grand pas de temps** provoque de l'erreur. Sur une orbite, la planète parcourt une portion significative de cercle en un pas — l'accélération change beaucoup entre le début et la fin du pas.

Le **sub-stepping** consiste à diviser le dt d'une frame visuelle en plusieurs sous-pas plus petits :

$$dt_{\text{sub}} = d \frac{t_{\text{frame}}}{N}$$

On calcule la physique N fois par frame, chaque fois avec un petit dt_{sub} . Le rendu visuel ne se fait qu'une fois — c'est la physique qui est affinée.

💡 Sub-stepping vs RK4 — les deux se complètent

- **RK4** améliore la **qualité** de chaque pas (4 évaluations au lieu d'1).
- **Sub-stepping** améliore la **taille** du pas (plus de pas plus petits).
- Ensemble, ils donnent une précision excellente même avec un `timeScale` élevé.

Sub-stepping en JavaScript.

```
function updatePhysics(dtFrame) {  
    const dtSubStep = dtFrame / params.subSteps;  
    for (let i = 0; i < params.subSteps; i++) {  
        for (const p of planets) {  
            if (params.integrator === 'Euler')  
                updatePhysicsEuler(p, dtSubStep);  
            else  
                updatePhysicsRK4(p, dtSubStep);  
        }  
        totalPhysicsTime += dtSubStep;  
    }  
}
```

💡 Combien de sub-steps ?

Avec 20 sub-steps et RK4, les orbites restent stables même à `timeScale = 1.5`. Sans sub-stepping (`subSteps = 1`), même RK4 dérape à haute vitesse. Testez dans le TP : baissez `subSteps` à 1 et augmentez `timeScale` — vous verrez les orbites se dégrader en temps réel.

Partie 6 : Le Système Solaire — Données

Les planètes

On utilise des unités **simulées** (pas les vraies unités SI) pour garder des nombres manipulables :

- $G = 100$ (constante gravitationnelle simulée)
- $M_{\text{soleil}} = 3330$ (masse du Soleil, en unités de masse terrestre $\times 10$)
- 1 UA = 18 unités de monde (pour que tout tienne à l'écran)

Planète	Masse ($M_{\oplus}$)	Distance (UA)	Couleur
Mercure	0.055	0.39	gris
Vénus	0.815	0.72	jaune
Terre	1.000	1.00	bleu
Mars	0.107	1.52	rouge
Jupiter	317.8	5.20	ocre
Saturne	95.2	9.54	beige

💡 Pourquoi les masses ne servent à rien (sans N-body)

Sans la gravité mutuelle entre planètes, la masse de chaque planète **n'intervient pas** dans le calcul — l'accélération $a = GM_{\text{soleil}}/r^2$ ne dépend que de la masse du Soleil. Les masses sont incluses pour le **bonus N-body**.

Partie 7 : TP — Missions

💡 Mise en place

Ouvrir `session12_solar.html` via Live Server. La scène, le Soleil, les planètes (meshes), les trails, le HUD et la GUI sont fournis — il reste **cinq fonctions** à compléter. Au départ, les planètes ne bougent pas : la vitesse orbitale renvoie 0 et l'accélération est nulle. C'est normal.

Mission 1 — `computeOrbitalVelocity(distWorld)`

Calculer la vitesse pour une orbite circulaire à la distance `distWorld` du Soleil :

$$v = \sqrt{G \cdot M_{\text{soleil}} / r}$$

Utiliser `G`, `SUN_MASS` et `distWorld`. **Indices** : `Math.sqrt()`, une seule ligne.

Test : les planètes doivent apparaître sur l'axe x et commencer à orbiter dans le sens trigonométrique (vers $-z$).

Mission 2 — `getAcceleration(position)`

Calculer l'accélération gravitationnelle du Soleil sur un objet à `position` (le Soleil est à l'origine) :

1. Direction : $-\vec{r}$ (vers le Soleil)
2. Distance : $r = \text{position.length}()$
3. Magnitude : $a = G \cdot M / r^2$
4. Retourner : direction normalisée $\times$ magnitude

Sécurité : si $r^2 < 5$, retourner un vecteur nul (éviter la singularité au centre du Soleil).

Indices : `position.lengthSq()`, `position.clone().negate().normalize()`, `multiplyScalar()`.

Mission 3 — `updatePhysicsEuler(p, dt)`

Implémenter l'intégration d'Euler explicite :

1. Calculer l'accélération à la position actuelle
2. $v + = a \cdot dt$
3. $x + = v \cdot dt$ (avec la **nouvelle** vitesse)

Test : sélectionner "Euler" dans la GUI. Les planètes orbitent mais **dérangent** — l'orbite grandit à chaque révolution. C'est le comportement attendu, la preuve qu'Euler ne convient pas.

Mission 4 — `updatePhysicsRK4(p, dt)`

Implémenter Runge-Kutta 4 (voir Partie 4). Les 4 étapes k_1, k_2, k_3, k_4 avec les positions/vitesses intermédiaires, puis la combinaison finale.

Pièges :

- Ne **pas** modifier `p.pos` et `p.vel` pendant le calcul — utiliser `.clone()`.
- L'accélération ne dépend que de la **position** (pas de la vitesse) — c'est `getAcceleration(x_intermédiaire)` à chaque étape.
- Les k_v et k_x sont déjà multipliés par dt — la combinaison finale divise par 6, pas par $6 \cdot dt$.

Test : sélectionner “RK4” dans la GUI. Les orbites restent **stables** même après 10+ révolutions. Comparer avec Euler — la différence est frappante.

Mission 5 — `updatePhysics(dtFrame)`

Implémenter la boucle de sub-stepping :

1. Calculer $dt_{\text{sub}} = dt_{\text{frame}} / \text{subSteps}$
2. Boucler `subSteps` fois : pour chaque sous-pas, intégrer toutes les planètes
3. Incrémenter `totalPhysicsTime` à chaque sous-pas
4. Choisir Euler ou RK4 selon `params.integrator`

Test : baisser `subSteps` à 1 et monter `timeScale` à 0.5 — les orbites se dégradent. Remonter `subSteps` à 20 — tout redevient stable.

Scénarios à observer

Expériences recommandées

Euler vs RK4 : avec `subSteps = 20`, basculer entre Euler et RK4. Euler dérape en quelques révolutions, RK4 reste stable indéfiniment.

Sub-steps : avec RK4, baisser `subSteps` de 20 à 1. À `timeScale = 0.05`, peu de différence. À `timeScale = 0.5`, RK4 sans sub-steps dérape — le sub-stepping est essentiel à haute vitesse.

Vitesse initiale : dans le code, multiplier `vOrbit` par 0.7 — la planète spirale vers le Soleil. Par 1.3 — la planète s'échappe en ellipse large. Par 1.0 — cercle parfait.

TimeScale extrême : monter `timeScale` à 1.5 avec RK4 + 20 sub-steps. Les années défilent en quelques secondes — on voit Mercure faire 10 tours pendant que Jupiter en fait 1.

Partie 8 : Bonus

Bonus 1 — Gravité mutuelle (N-body)

Activer `params.nBody = true` dans la GUI. Dans `getAcceleration`, ajouter l'attraction de chaque autre planète :

$$\vec{a}_{\text{total}} = \vec{a}_{\text{soleil}} + \sum_i G \cdot \frac{m_i}{r_i^2} \cdot \hat{u}_i$$

Observation : les orbites ne sont plus parfaitement circulaires — les planètes se perturbent mutuellement. Jupiter (la plus massive) dévie Mars et la Terre. C'est ainsi que Neptune a été **découvert** en 1846 — par les perturbations qu'il causait sur Uranus, avant même d'être observé au télescope !

Bonus 2 — Anneaux de Saturne

Ajouter un anneau à Saturne avec `THREE.RingGeometry`. Faire tourner l’anneau à une vitesse légèrement différente de Saturne (les anneaux ne sont pas solides — chaque particule orbite à sa propre vitesse).

Bonus 3 — Lune de la Terre

Ajouter une Lune qui orbite autour de la Terre. La Lune subit la gravité du Soleil **et** celle de la Terre. Il faut calculer l’accélération totale et intégrer la Lune comme une planète supplémentaire — mais sa position initiale est relative à la Terre, pas au Soleil.

Bonus 4 — Comparaison énergétique

Calculer l’énergie totale de chaque planète ($E = 1/2v^2 - GM/r$) et l’afficher dans le HUD. Avec Euler, l’énergie **augmente** progressivement. Avec RK4, elle reste **constante** — c’est la preuve que RK4 respecte la conservation d’énergie.

Bonus 5 — Post-processing et rendu

La solution utilise trois techniques de rendu pour un look “space sim” :

- **Bloom** (`UnrealBloomPass`) : les éléments lumineux (Soleil, trails) débordent en halo doux. Toggle dans la GUI.
- **Trails à dégradé** : au lieu d’une ligne uniforme, chaque trail utilise des `vertexColors` avec un alpha qui croît du tail (invisible) vers le head (vif). Combiné avec l’`AdditiveBlending`, les trails “glow”.
- **Halo du Soleil** : un `Sprite` avec une texture de gradient radial additif donne un halo diffus autour du Soleil.

Pour aller plus loin : ajouter un `FilmPass` (grain de film), un `Vignette` via shader, ou des étoiles scintillantes dans le fond HDR.

Synthèse

Ce qu'il faut retenir

- **Euler** est simple mais **accumule de l’erreur** sur les systèmes conservatifs (orbites) — l’énergie dérive.
- **RK4** évalue l’accélération à 4 endroits du pas de temps — précision $O(dt^4)$ au lieu de $O(dt)$.
- **Sub-stepping** divise le pas de temps en N sous-pas — indispensable à haute vitesse même avec RK4.
- La **vitesse orbitale** $v = \sqrt{GM/r}$ donne une orbite circulaire parfaite.
- **RK4 + sub-stepping** est le standard pour les simulations orbitales en temps réel.
- Pour des simulations scientifiques (NASA, ESA), on utilise des méthodes encore plus précises (symplectiques, Verlet leapfrog) qui **garantissent** la conservation d’énergie sur des millions de pas.

💡 Et dans les jeux ?

Les jeux n'ont généralement **pas besoin** de RK4 — les orbites sont rares et le sub-stepping d'Euler suffit pour la plupart des effets. RK4 est pertinent pour les simulations spatiales (Kerbal Space Program, Elite Dangerous) où la précision orbitale est le cœur du gameplay.